数据结构 A 第 5 次作业

姓名: 王子川　　**学号:** PB25000055

1 (3.27(1)) Ackermann 函数

根据 Ackermann 函数的递归定义，可以直接写出如下递归算法.

```
int ack(int m, int n) {
    if (!m) return n + 1;
    if (!n) return ack(m - 1, 1);
    return ack(m - 1, ack(m, n - 1));
}
```

2 (3.31) 判断回文串

思路是顺序扫描字符串 s, 让除了结束符 @ 外的元素依次入栈 S 和入队 Q. 依次让 S 中的栈顶元素 a 出栈, 让 Q 中的队首元素 b 出队. 因为栈是 LIFO 而队列是 FIFO, 因此每次比较的 a, b 是关于字符串中心对称的.

若存在某个位置 $a \neq b$, 说明 s 不是回文串, 返回 `FALSE`; 否则若 $a = b$ 对所有位置都成立, 则说明 s 是回文串, 返回 `TRUE`.

```
bool Palindrome(string s) {
    Stack S; SqQueue Q;
    InitStack(S);
    InitQueue(Q);

    int i = 0;
    while (s[i] != '@') {
        char c = s[i++];
        Push(S, c);
        EnQueue(Q, c);
    }

    // Compare the reverse order with the original order.
    char a, b;
    while (!StackEmpty(S)) {
        Pop(S, a);
        DeQueue(Q, b);
        if (a != b)
            return FALSE;
    }

    return TRUE;
}
```

设字符串长度为 n. 每个字符至多出入栈队各一次, 因此时空复杂度均为 $O(n)$.

3 (3.32) 循环队列求 Fibonacci 数列

考虑设计一个容量为 k 的循环队列. 先把 $f_0, f_1, \ldots, f_{k-2}$ 设为 0, f_{k-1} 设为 1. 维护一个变量 `sum` 表示当前循环队列中 k 个元素之和. 根据 k 阶 Fibonacci 数列的定义, 它同时也是下一个元素的值.

每次循环的过程中, 先判断下一个元素的值 `sum` 是否大于 `max`. 若是, 根据题意, 循环应当终止, 算法返回 OK. 否则, 记原队首元素为 `old`, 用 `sum` 覆盖该元素, 并将队首和队尾位置同时循环后移一位. 此时队列中 k 个元素之和变为 $2\texttt{sum} - \texttt{old}$.

由于循环队列始终保存恰好 k 个元素, 这里 $Q.front = Q.rear$ 表示队列已满.

```cpp
Status FibonacciQueue(SqQueue& Q, int k, int max) {
    if (k <= 1) return ERROR;

    Q.front = 0;
    Q.rear = 0;

    // Initialize f(0), ..., f(k-2) = 0, f(k-1) = 1.
    for (int i = 0; i < k - 1; i++)
        Q.base[i] = 0;
    Q.base[k - 1] = 1;

    // The sum of the current k terms is the next term.
    long long sum = 1;

    while (1) {
        if (sum > max) return OK;

        // Replace the oldest term by the new term.
        long long old = Q.base[Q.front];
        Q.base[Q.front] = sum;

        Q.front = (Q.front + 1) % k;
        Q.rear = (Q.rear + 1) % k;

        // Update the sum of the k terms.
        sum = 2 * sum - old;
    }

    return OK;
}
```

初始化需要 $O(k)$ 时间, 此后每次循环的时间复杂度为 $O(1)$. 若 $f_n \le \texttt{max} < f_{n+1}$, 则算法的总时间复杂度为 $O(n)$, 空间复杂度为 $O(k)$.